LAB # 3

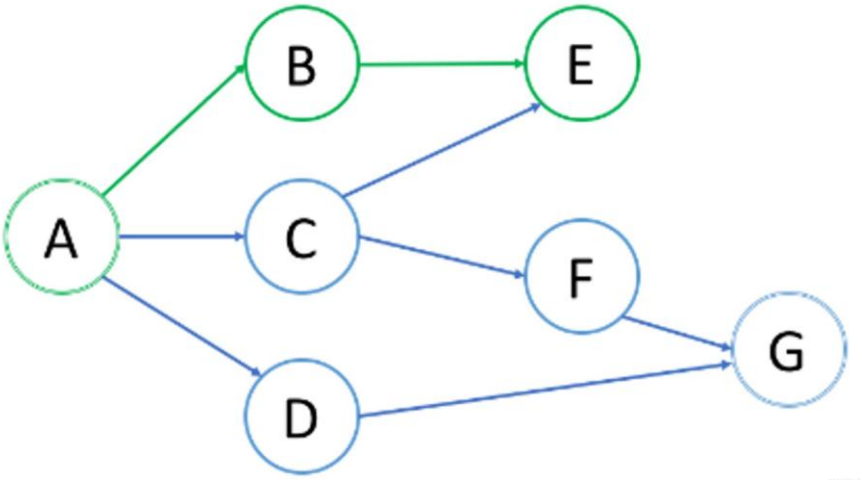
NAME: Hifza Hussain

ROLL NO: SU92-BSAIM-F25-330

SECTION: BSAI-3A

Task 1: Manual Tracing of BFS and DFS

Original Graph



The graph is represented using the following list:

```
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["G"],
    "F": [],
    "G": []
}
```

Start Node: A

Goal Node: G

Successor order: Left to right

Graph type: Directed graph

Breadth-First Search (BFS)

BFS Trace:

Step	Processed Node	Queue After Processing	Path
0	—	A	—
1	A	B,C	A
2	B	C,D,E	A → B
3	C	D,E,F	A → C
4	D	E,F	A → B → D
5	E	F,G	A → B → E
6	F	G	A → C → F
7	G	Empty	A → B → E → G

BFS Results

- Processing Order: A, B, C, D, E, F, G
- Visit Order: A, B, C, D, E, F, G
- Final Path: A → B → E → G
- Path Length: 3 edges
- Goal Found: Yes

BFS code:

```python
from collections import deque
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["G"],
    "F": [],
    "G": []
}
def bfs(start, goal):
    q = deque([(start, [start])])
    seen = {start}
    while q:
        node, path = q.popleft()
        if node == goal:
            return path
        for n in graph[node]:
            if n not in seen:
                seen.add(n)
                q.append((n, path + [n]))
    return None
print(bfs("A", "G"))
```

BFS Output

```
Problems  Output  Debug Console  ...  Filter (e.g. text, !excludeText, t...
[Running] python -u "c:\Users\DELL\Documents\ai lab\task 3\3.py\bfs.py"
['A', 'B', 'E', 'G']

[Done] exited with code=0 in 0.139 seconds
```

3 Depth-First Search (DFS)

DFS Trace

The supplied DFS implementation pushes successors in reverse order so the first successor is processed first.

Step	Processed Node	Stack (Bottom → Top)	Path
0	—	A	—
1	A	C , B	A
2	B	C, E, D	A → B
3	D	C, E	A → B → D
4	E	C, G	A → B → E
5	G	C	A → B → E → G

DFS Results

- Processing Order: A, B, D, E, G
- Visit Order: A, B, D, E, G
- Final Path: A → B → E → G
- Path Length: 3 edges
- Goal Found: Yes

DFS Python Code

```python
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["G"],
    "F": [],
    "G": []
}
def dfs(start, goal):
    stack = [(start, [start])]
    seen = set()
    while stack:
        node, path = stack.pop()
        if node in seen:
            continue
        seen.add(node)
        if node == goal:
            return path
        for n in reversed(graph[node]):
            if n not in seen:
                stack.append((n, path + [n]))
    return None
print(dfs("A", "G"))
```

DFS Output:

```
Problems  Output  Debug Console  ...  Filter (e.g. text, !excludeText, t...

[Running] python -u "c:\Users\DELL\Documents\ai lab\task 3\3.py\DFS.PY"
['A', 'B', 'E', 'G']

[Done] exited with code=0 in 0.073 seconds
```

Conclusion:

BFS and DFS both successfully reached goal G. BFS explored all nodes level by level, whereas DFS followed the branch A-B-D before backtracking to E. Both algorithms returned A-B-E-G in this graph.

Task 2: Implementation of DLS and IDDFS

Depth-Limited Search (DLS)

DLS uses three possible statuses:

- FOUND: Goal node is successfully found.
- CUTOFF: Search is stopped because the depth limit is reached.
- FAILURE: No solution exists in the explored search space.

DLS Python Code

DLS.PY > ...

```python

graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["G"],
    "F": [],
    "G": []
}

FOUND = "FOUND"
CUTOFF = "CUTOFF"
FAILURE = "FAILURE"

def dls(start, goal, limit):
    processed = []
    path = [start]

    def visit(node, depth):
        processed.append(node)

        if node == goal:
            return FOUND

        if depth == limit:
            if graph[node]:
                return CUTOFF
            return FAILURE

        cutoff_occurred = False

        for child in graph[node]:
            if child in path:
                continue

```

```
            path.append(child)
            result = visit(child, depth + 1)

            if result == FOUND:
                return FOUND

            path.pop()

            if result == CUTOFF:
                cutoff_occurred = True

        if cutoff_occurred:
            return CUTOFF

        return FAILURE

    status = visit(start, 0)

    return status, path[:] if status == FOUND else None, processed

for limit in range(1, 5):
    status, path, order = dls("A", "G", limit)

    print("Limit:", limit)
    print("Status:", status)
    print("Path:", path)
    print("Processed:", order)
    print()
```

DLS Output and Results:

Limit	Status	Processing Order	Path
1	CUTOFF	A, B, C	Not found
2	CUTOFF	A, B, D, E, C, F	Not found
3	FOUND	A, B, D, E, G	A-B-E-G
4	FOUND	A, B, D, E, G	A-B-E-G

Explanation

At limit 1, the algorithm cannot explore deeper than depth 1.

At limit 2, the algorithm cannot reach G because G exists at depth 3.

At limit 3, the goal G is successfully found.

At limit 4, the goal is still found at depth 3, so no additional depth is needed.

Iterative Deepening Depth-First Search (IDDFS)

IDDFS Code:

IDDFS > ...

```
1
2   def iddfs(start, goal, max_depth):
3       all_orders = []
4
5       for limit in range(1, max_depth + 1):
6           status, path, order = dls(start, goal, limit)
7
8           all_orders.append((limit, status, path, order))
9
10          if status == FOUND:
11              return status, path, all_orders
12
13      return CUTOFF, None, all_orders
14
15
16  for maximum in range(1, 5):
17      status, path, runs = iddfs("A", "G", maximum)
18
19      print("Maximum Depth:", maximum)
20      print("Status:", status)
21      print("Path:", path)
22      print("Limits Attempted:", [r[0] for r in runs])
23      print()
24
```

IDDFS Results:

Maximum Depth	Limits Attempted	Status	Path
1	1	CUTOFF	None
2	1,2	CUTOFF	None
3	1, 2, 3	FOUND	A-B-E-G
4	1, 2, 3	FOUND	A-B-E-G

IDDFS restarts the search from the root node for every depth limit. Therefore, nodes near the root are visited multiple times.

For example:

- Limit 1: A is explored.
- Limit 2: A is explored again.
- Limit 3: A is explored again.

This repeated work increases processing but allows IDDFS to use less memory than BFS.

The shortest path from A to G contains three edges:

A → B → E → G

Therefore, G exists at depth 3.

Depth limits 1 and 2 cannot reach G, so they return CUTOFF instead of FOUND.

Task 3: Weighted Graph and Uniform Cost Search (UCS)

UCS finds the minimum-cost path when all edge weights are non-negative.

Weighted Graph

According to the original graph is retained and an additional corridor C → G is added.

Edge Weights:

Edge	Weights
A → B	1
A → C	4
B → D	2
B → E	1
C → F	3
E → G	1
C → G	8

The added corridor is:

C → G = 8

The alternative route A → C → G has cost:

4 + 8 = 12

The route A → B → E → G has cost:

1 + 1 + 1 = 3

UCS Python Code:

UCS.PY > ucs

```python

import heapq
weighted_graph = {
    "A": [("B", 1), ("C", 4)],
    "B": [("D", 2), ("E", 1)],
    "C": [("F", 3), ("G", 8)],
    "D": [],
    "E": [("G", 1)],
    "F": [],
    "G": []
}

def ucs(start, goal):
    frontier = [(0, start, [start])]
    best_cost = {start: 0}
    processed = []

    while frontier:
        cost, node, path = heapq.heappop(frontier)

        if cost != best_cost.get(node):
            continue

        processed.append((node, cost))

        if node == goal:
            return path, cost, processed

        for child, weight in weighted_graph[node]:
            new_cost = cost + weight

```

UCS.PY > ucs

```python
def ucs(start, goal):

            if new_cost < best_cost.get(child, float("inf")):
                best_cost[child] = new_cost
                heapq.heappush(
                    frontier,
                    (new_cost, child, path + [child])
                )

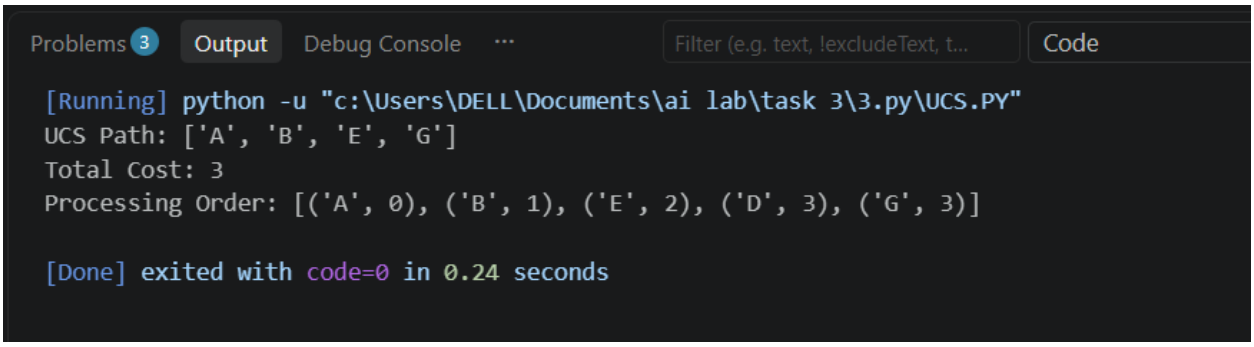
    return None, float("inf"), processed

path, cost, order = ucs("A", "G")

print("UCS Path:", path)
print("Total Cost:", cost)
print("Processing Order:", order)

```

UCS Output:



UCS Manual Trace:

Step	Processed Node	Cost	Priority Queue (Node, Cost)
0	**—**	**—**	**A(0)**
1	**A**	**0**	**B(1), C(4)**
2	**B**	**1**	**E(2), D(3), C(4)**
3	**E**	**2**	**D(3), G(3), C(4)**
4	**D**	**3**	**G(3), C(4)**
5	**G**	**3**	**C(4)**

The goal G is found with total cost 3.

UCS Results

- Processing Order: A, B, E, D, G
- Final Path: A → B → E → G
- Number of Edges: 3
- Total Cost: 3
- Goal Found: Yes

BFS and UCS Comparison

BFS ignores edge weights and selects a path with the fewest edges.

On the extended weighted graph:

- BFS selects A → C → G.

- BFS path cost = 4 + 8 = 12.
- BFS number of edges = 2.

UCS selects A → B → E → G.

- UCS path cost = 1 + 1 + 1 = 3.
- UCS number of edges = 3.

Algorithm	Path	Edges	Total Cost
BFS	**A-C-G**	**2**	**12**
UCS	**A-B-E-G**	**3**	**3**

A path with fewer edges does not necessarily have a lower total cost. BFS considers the number of edges, whereas UCS considers the sum of their weights.

Therefore, BFS may choose a shorter route in terms of hops that is more expensive, while UCS chooses the route with minimum total cost.

Failure Case and Edge Case Analysis

Failure Case

Suppose the edge E → G is removed and the added corridor C → G is also removed.

The graph no longer contains any route from A to G.

Expected Behavior

Algorithm	Expected Result
BFS	**Returns None**
DFS	**Returns None**
DLS	**FAILURE when search is exhausted**
IDDFS	**Eventually FAILURE when the finite graph is fully explored**
UCS	**Returns None / infinite cost**

Observed Behavior

The algorithms explore all reachable nodes but cannot reach the goal.

BFS and DFS terminate after exploring the reachable graph.

DLS distinguishes between a depth cutoff and a genuine failure. A cutoff means more depth may help; failure means no available path remains in the explored search space.

Possible Cause

The goal node is disconnected from the reachable portion of the graph.

Proposed Improvement

- Validate whether the start and goal nodes exist.
- Check graph connectivity.
- Avoid revisiting nodes unnecessarily.
- Distinguish CUTOFF from FAILURE.
- Handle missing graph keys safely.
- Display a clear message when no path exists.

Reflection

During this lab, I implemented and compared different search algorithms.

I learned that BFS uses a queue and explores nodes level by level, while DFS uses a stack and explores deeper branches first.

DLS restricts the search to a specified depth, whereas IDDFS gradually increases the depth limit. IDDFS repeats some work but reduces memory requirements.

UCS uses edge costs to select the least expensive path. The experiment demonstrated that the shortest path in terms of hops may not be the cheapest path.

I also learned the importance of correct successor ordering, visited-node handling, cycle checking, and distinguishing search cutoff from failure.

Responsible Practice

- Use original or properly cited code.
- Document graph structure and edge weights.
- Provide reproducible inputs and outputs.
- Avoid uploading credentials or private information.

- Test algorithms on different graph configurations.
- Clearly explain assumptions and limitations.
- Verify computational results independently.

Final Conclusion

This lab demonstrated the practical working of BFS, DFS, DLS, IDDFS, and UCS.

BFS and DFS successfully found the goal in the original graph. DLS and IDDFS demonstrated the importance of depth limits. UCS identified the minimum-cost route in the weighted graph.

The experiment confirmed that different search algorithms use different strategies and that their performance depends on graph structure, depth, and edge costs.